

KNOWLEDGE TRANSFER DOCUMENT

RageV

Engine architecture, rendering engineering,
and the order it should be built in

A Windows game engine with two rendering backends, a scene editor, two scripting languages and a shipped game. This document explains what it is made of, how the pieces connect, what happens in a frame — and, most of all, the sequence in which a competent engineer should build these things so that each step is provable before the next one leans on it.

Subject RageV — C++20, Vulkan 1.3 and OpenGL 4.6, CMake

Audience Software engineers. No graphics background assumed; C++ fluency is.

Status at time of writing Phases 0–5 complete, phase 6 in progress. 888 automated checks.

Contents

0 How to read this

Who it is for · What it assumes · The one idea

1 What the engine is

Capability · Scale · The two applications

2 The build order — the spine of the document

- 2.1 Why order is the hard part
- 2.2 Phase 0 — the scene model, before anything is drawn
- 2.3 Phase 1 — assets and identity
- 2.4 Phase 2 — the loop, physics, scripts, audio
- 2.5 Phase 3 — the renderer, abstraction first
- 2.6 Phase 4 — projects, runtime, packaging
- 2.7 Phase 5 — scripting for real
- 2.8 The interlude that matters most — build a game
- 2.9 Phase 6 — what the game proved was missing

3 Architecture

- 3.1 Layers and the direction of dependency
- 3.2 The module boundary — engine as a DLL
- 3.3 The three "one description" systems

4 The frame

- 4.1 Fixed simulation, free rendering
- 4.2 The render path, pass by pass
- 4.3 The render graph

5 Rendering engineering

- 5.1 Designing an RHI over two very different APIs
- 5.2 Frames in flight, and everything that follows from them
- 5.3 Batching and instancing
- 5.4 Lighting: clustered forward
- 5.5 Shadows
- 5.6 Image-based lighting and reflection probes
- 5.7 Post-processing
- 5.8 Transparency, and why it is the hardest easy problem
- 5.9 Compute, and moving a simulation to the GPU

6 The systems around the renderer

- 6.1 Scene, entities, reflection
- 6.2 Assets
- 6.3 Scripting in two languages
- 6.4 Physics and audio

7 What experience taught — the load-bearing invariants

8 Verification: how any of this is known to work

9 Where it stands, and what comes next

A Appendix — repository layout, build, tools

0 How to read this

This is not an API reference and not a tour of a finished thing. It is an attempt to answer a harder question: if you had to build this, what would you build first, and what would you have to understand at each step to avoid painting yourself into a corner?

Who it is for

Somebody with real software engineering experience — comfortable in C++, comfortable with build systems, comfortable reading someone else's architecture — who has not built a renderer or an editor before. Graphics knowledge is not assumed; where a graphics concept is needed it is explained in the place it first matters, rather than in a chapter of preliminaries you would have forgotten by the time it was used.

What it assumes about the reader's goal

That you want a working engine rather than a rendering demo. Those are different projects and they diverge almost immediately. A rendering demo can hard-code its scene, load its assets from a fixed path, and put its camera in a global. An engine cannot do any of those things, and the difference is not a refactor you do later — it is the reason the build order below starts where it does.

The one idea, if you read nothing else

Build the thing that other things will be described in terms of, before you build the things. Identity before serialization. Serialization before the editor. Reflection before the inspector. The hardware abstraction before the renderer. The render graph before the passes. Every one of those pairs is cheap in that order and brutally expensive in the other, and the expense is not the code — it is that the second one has to be right in a hundred call sites you have already written.

The corollary is that the most valuable early work looks the least impressive. Weeks where the screen shows nothing new are the weeks that decide whether the project has a ceiling. This document tries to make those weeks legible.

A note on the code excerpts

Every snippet is taken verbatim from the engine, including its comments. The comments are load-bearing: in this codebase a comment usually records *why* a thing is not the obvious thing, and most of them exist because the obvious thing was tried first and failed in a way that took days to find. Where that is the case, the failure is described too.

1 What the engine is

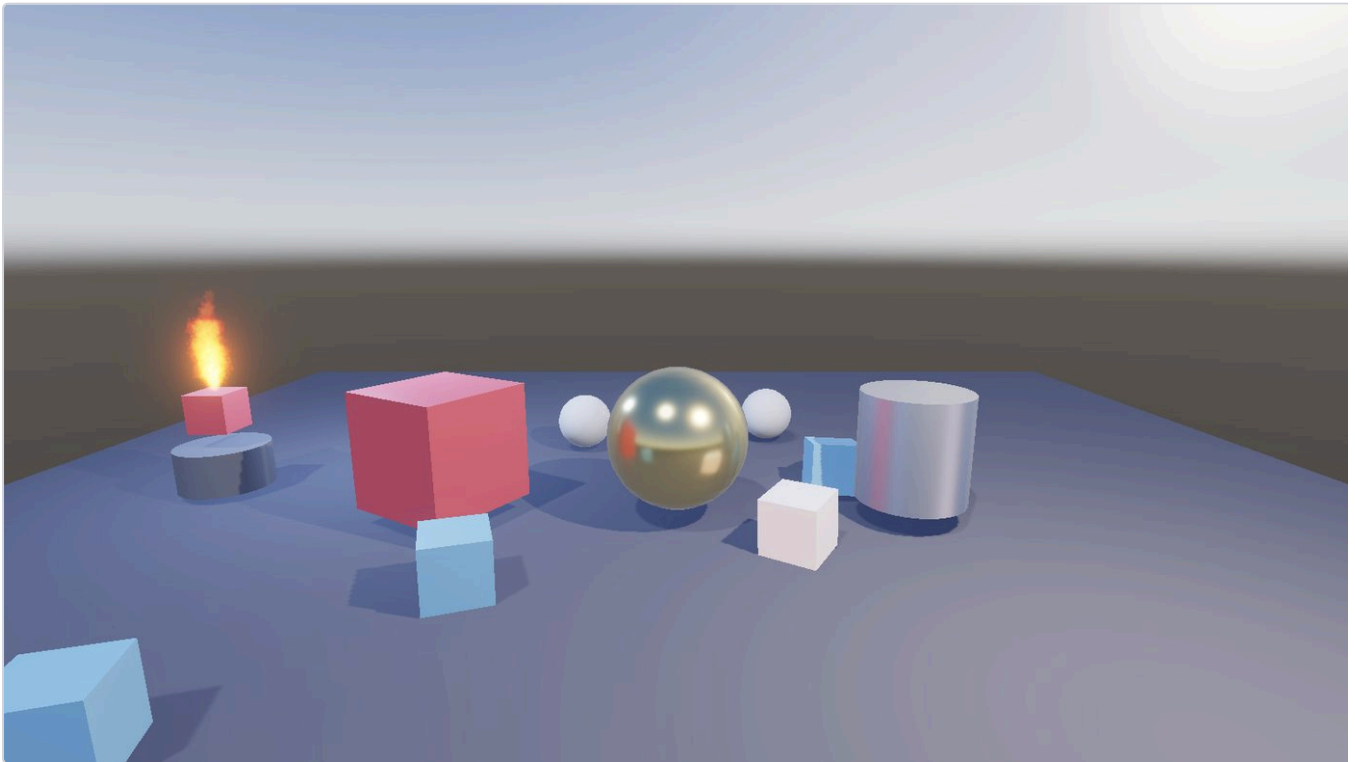


Figure 1. The sample project, rendered by the standalone runtime. Physically-based shading, a gold sphere reflecting a real-time reflection probe, cascaded shadow maps, image-based lighting from an HDR environment, and a GPU-textured particle flame on the pedestal. The boxes are Jolt rigid bodies that have fallen and settled.

Capability

RageV is a general-purpose 3D engine with an editor, targeting Windows. It renders through **either Vulkan 1.3 or OpenGL 4.6**, chosen at startup, from one set of engine code and one set of shaders. It has physically-based shading with image-based lighting, cascaded and omnidirectional shadows, real-time reflection probes, clustered forward lighting with no light count limit, frustum culling, instanced batching, skeletal animation, a CPU and GPU particle system, HDR post-processing, rigid-body physics, positional audio, and scripting in **both C++ and C#** with live reload in each.

It ships games: a project is a folder with its own assets, scenes, scripts and game module, and the editor can package one into a standalone folder that runs without the editor present.

Scale

Dimension	Figure	Note
Rendering backends	2	Vulkan 1.3 (dynamic rendering, no <code>VkRenderPass</code>) and OpenGL 4.6
Scripting languages	2	C++ game module DLL; C# via CoreCLR, both with hot reload
Automated checks	888	Run against both backends on every change
Frame, sample scene	0.80 ms Vulkan / 0.97 ms OpenGL	Release, 1600×900, vsync off
Frame, 1000 meshes	1.88 ms Vulkan / 1.59 ms OpenGL	Instanced batching; OpenGL wins here, and that is discussed
Frame, 256 local lights	3.15 ms Vulkan / 3.48 ms OpenGL	Clustered; 5.56 / 7.03 ms unclustered

The two applications

The engine is a DLL. Two executables link it: **RageVEDitor**, which has panels, gizmos, an inspector and play-in-editor; and **RageVRuntime**, which opens a project and plays it. They share the entire frame-building path and differ only in where the finished image is delivered — an imported viewport texture for the editor, the backbuffer for the runtime. Keeping that difference to one variable is what stops the two drifting into two renderers.

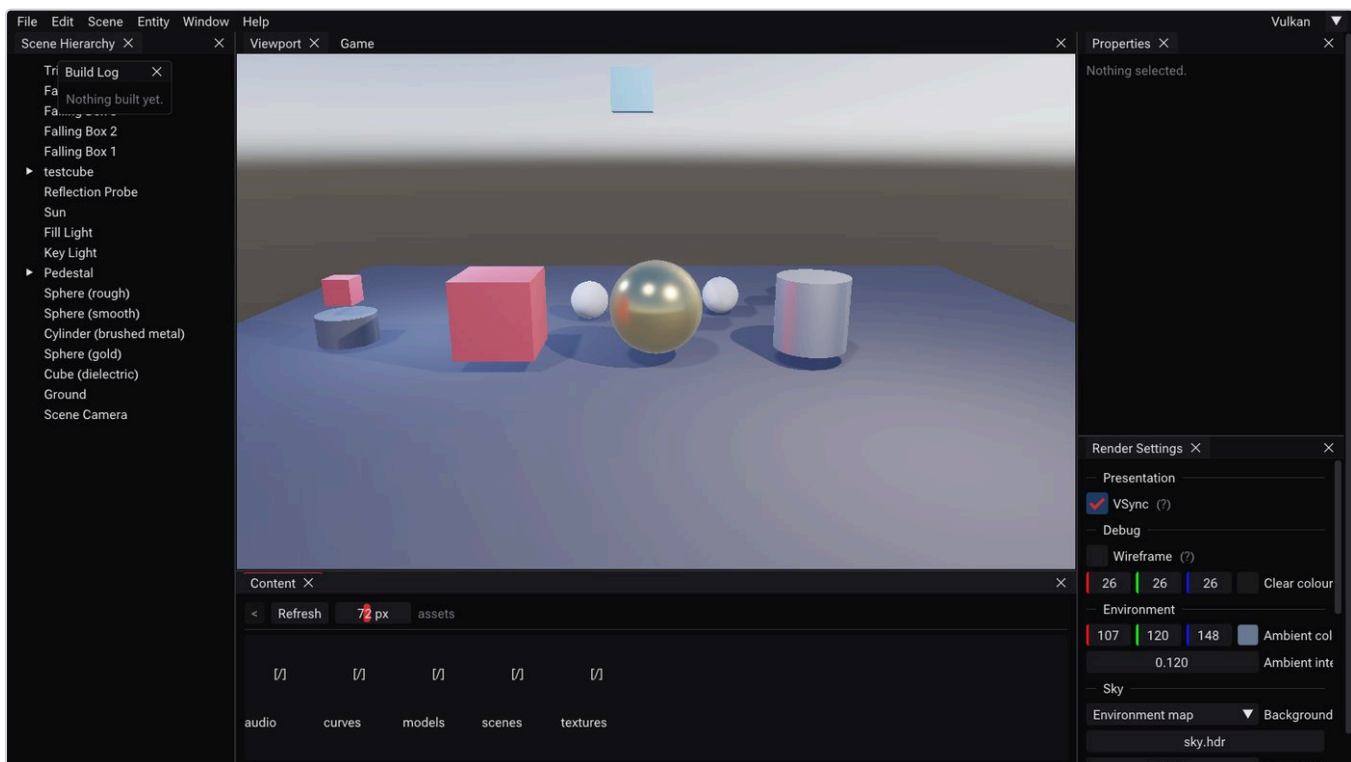


Figure 2. The editor. Scene hierarchy, viewport with a transform gizmo, reflection-driven inspector, content browser and render settings. Every field in the inspector is generated from the same component description that drives the scene file format and the C# bridge — see §3.3.

2 The build order

This is the part worth arguing with. Everything else in the document describes a state; this describes a sequence, and the sequence is the transferable knowledge.

2.1 Why order is the hard part

Most engine tutorials are organised by subsystem: here is a renderer, here is a physics integration, here is an editor. That organisation is fine for explaining a subsystem and actively misleading as a plan, because it hides the dependencies that actually bite. Those are rarely compile-time dependencies. They are *format* and *identity* dependencies, and they bite when you try to change something after a hundred files already contain it.

Two rules produced the order below.

Rule 1 — describe before you draw

An engine is a program for describing scenes. Rendering is one consumer of that description; serialization, the editor, the undo stack, prefabs, the network layer you have not written, and every script are the others. Build the description first, make it persist losslessly, and only then draw it. A renderer built first will invent its own notion of a scene, and every later system will be a translation layer.

Rule 2 — abstraction before the features that would hard-code themselves

There are exactly two places where retrofitting an abstraction costs more than writing it up front: the graphics API boundary and the frame's pass structure. Both are touched by every rendering feature you will ever add. Write the RHI before the second backend is a thought, and write the render graph before the third pass. Anywhere else, abstract late.

2.2 Phase 0 — the scene model, before anything is drawn

Deliverable: a scene you can build in an editor, save, load, and get back byte-identical, with undo.

Concretely, in this order:

1. **An editor camera.** Trivially small, and it means every later step is visible.
2. **Entity UUIDs.** Not the ECS library's handles — those are recycled and mean nothing outside one registry instance. A stable 64-bit identity on every entity, assigned at creation, preserved through save, load and play mode. *This is the single highest-leverage decision in the project* and it costs an afternoon. Retrofitting identity later means touching prefabs, scripts, the undo stack, the physics body map and every serialized reference at once.
3. **Transform hierarchy**, with parent stored by UUID and children rebuilt on load. Derived state that can disagree with its source is the most common bug class in a scene graph; storing only the parent link makes disagreement impossible.
4. **Component reflection** — a registry that describes each component's fields. Then write the serializer *against the registry*, and the inspector against it afterwards. §3.3 covers why this single decision pays for

itself repeatedly.

5. **An undo/redo command stack.** Early, because commands are how the editor mutates the scene, and retrofitting that means rewriting every editor action.
6. **A lossless round-trip test.** Build a scene, serialize, deserialize, serialize again, compare the strings. This one test will catch more real bugs over the project's life than any other, and it is twenty lines.

Why not draw first? Because the moment you can draw, you will want to draw more, and everything you add will be described in whatever ad-hoc way the renderer already understands. The scene model is the vocabulary. Fix the vocabulary first.

2.3 Phase 1 — assets and identity

Deliverable: a scene file that references a mesh without containing it.

Assets get the same treatment entities did: a 64-bit `AssetHandle`, a registry that maps handles to paths, and a `.meta` sidecar beside each source file that records the handle so it survives a rename. Then a real importer — glTF, via `cglTF` — and meshes, materials and textures as addressable assets rather than things a loader hands you.

The ordering point: **import comes after handles**. If you write the importer first it will return objects, and you will spend the next phase converting object returns into handle returns everywhere.

2.4 Phase 2 — the loop, physics, scripts, audio

Deliverable: a scene that *runs*, and reverts perfectly when you stop.

1. **The fixed-timestep loop first.** Before physics, before scripts. It is the contract both will be written against, and §4.1 explains why it cannot be retrofitted cheaply.
2. **Play/edit split with snapshot restore.** Play mode operates on a copy; stopping restores the original exactly. Doing this before scripts exist means scripts never get to assume they can mutate the authored scene.
3. **Input as named actions**, never key codes. A script written against `"Jump"` survives rebinding; one written against a key code does not, and by the time you notice, fifty scripts have key codes in them.
4. **Scripts, then physics, then contact callbacks.** In that order within the step, and documented as such — see §6.4.
5. **Audio** last of the four, because it is the only one nothing else depends on.

2.5 Phase 3 — the renderer, abstraction first

Deliverable: the picture in Figure 1, on two backends, from one set of code.

This is the largest phase and the one most likely to be attempted in the wrong order. The correct order is counter-intuitive because it front-loads two pieces of pure plumbing that render nothing new.

1. **The RHI.** A device, command list, buffer, texture, pipeline, shader, resource set and sampler abstraction, with two implementations. Writing this against a single backend is a trap: you will abstract the wrong things, because the shape of the API you have is invisible to you until a second one disagrees with it. §5.1 covers what to abstract and — as importantly — what not to.

2. **The render graph.** Passes declare the targets they read and write; the graph allocates and pools them and orders the passes. Written when the frame had three passes, which felt premature; the frame now has eleven, and adding a pass is a declaration rather than a surgery. §4.3.
3. **Then features,** in roughly this order, because each depends on the previous: cube maps and sky → environment reflections → image-based lighting → shadows → culling → instanced batching → clustered lighting → skeletal animation.

The measurement discipline starts here. Two of the optimisations in this phase — frustum culling and instanced batching — cut draw calls dramatically and produced *no* frame-time improvement whatsoever. Culling took 144 draws to 60; instancing took 3238 to 854; the clock did not move either time. A draw count is not a frame time. See §8.

2.6 Phase 4 — projects, runtime, packaging

Deliverable: a folder you can send someone that runs without the editor.

Doing this before the engine is "finished" is deliberate. Packaging is where you discover that your asset paths were relative to the editor's working directory, that your shaders were loaded from the source tree, and that the runtime renders a blank window because a pass clears the backbuffer after the scene is drawn — a bug with a perfectly healthy draw-call count, which is why the RHI has a screenshot capture (§8).

2.7 Phase 5 — scripting for real

Deliverable: the same game logic, in C++ or C#, either reloadable without restarting.

C# is hosted through CoreCLR, with the managed assembly loaded from *bytes* into a collectible `AssemblyLoadContext` that is retired and replaced on every build. The unload is verified with a weak reference; a context that will not die is reported rather than ignored, because a leaked context means the old code is still running and the symptom is "my change did nothing".

2.8 The interlude that matters most — build a game

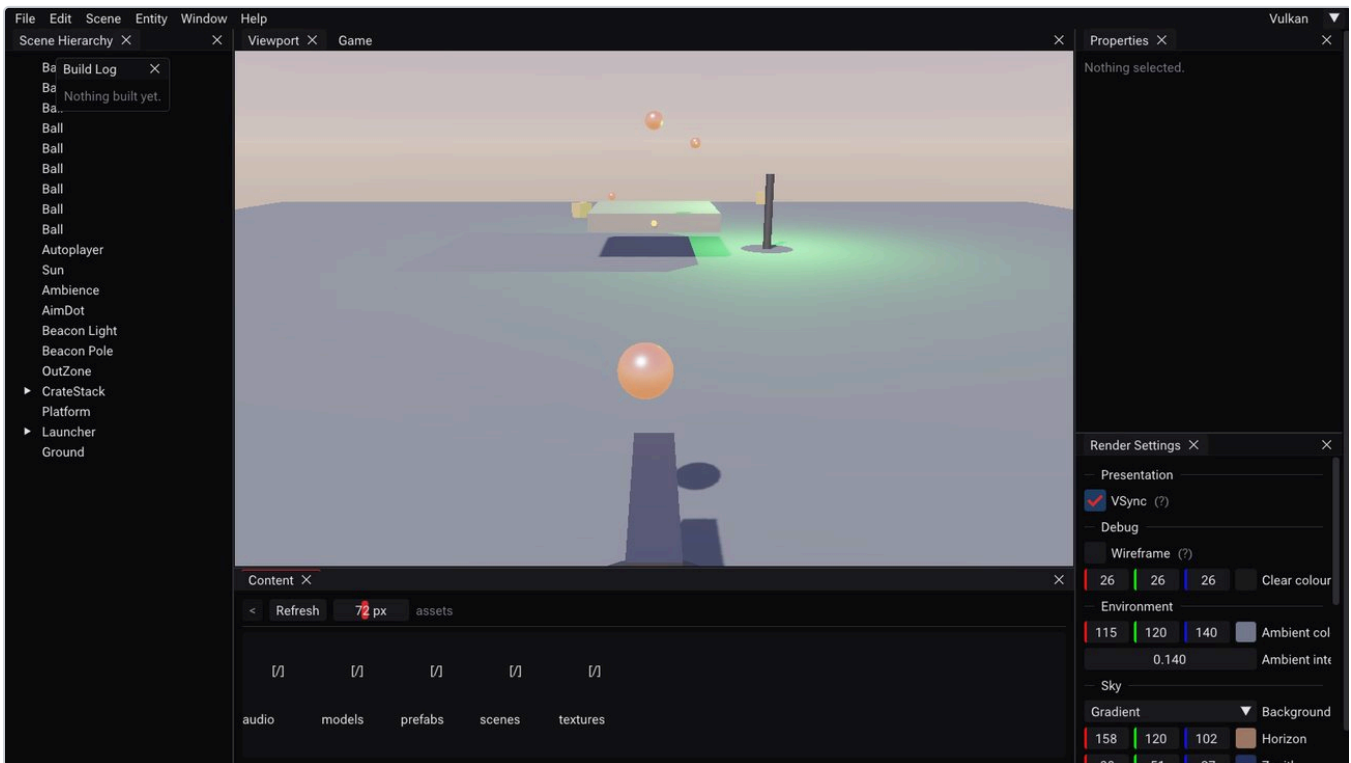


Figure 3. *Knockdown* — the mini game built to find out what the engine was missing. Aim a launcher, lob balls at a crate pyramid, knock them into the scoring zone. Deliberately split across both languages: C++ does input, aiming and spawning; C# does scoring, the component bridge and audio.

Building a small, complete, shippable game before adding more engine features is the highest-value scheduling decision in the whole project. It found **six real engine holes in one session**, every one of which was invisible from inside the engine:

- A menu item that had been implemented but never wired into the menu.
- A rigid body added during play never joined the simulation — `AddBody` existed and had no callers. The first ball fired hung in the air.
- C# collision callbacks were never delivered from a real simulation. The entire managed contact surface was written, bound, documented and tested at the table, and unreachable.
- Nothing outside the test suite ever booted the .NET host, so in a fresh editor C# was dead until a manual build, and a packaged game had no C# at all.
- Editor C# builds always failed, because a reference path was resolved against the wrong directory.
- A deferred-deletion race that freed descriptor sets while a frame was still in flight — found later, but by the churn the game created.

Note the pattern: *none of these were bugs in a feature*. They were bugs in the seams between features, and only an application that crosses the seams finds them. A test suite written by the same person who wrote the feature tends to test the feature.

2.9 Phase 6 — what the game proved was missing

The game produced a ranked list of what actually hurt, which is a far better backlog than a list of what sounds impressive. In order: **no text or UI** (the game communicated entirely through a light and four sounds), **no**

per-frame script hook (camera smoothing and recoil had nowhere to live), **no particles**, and some small API gaps in audio.

Particles were built first at the user's direction, and are covered in §5.8 and §5.9. Text and UI, and the per-frame hook, remain open.

3 Architecture

3.1 Layers and the direction of dependency

```
Project (.rvproject: asset root, start scene, fixed Hz)
|
Application (fixed-step loop, InputMap, AssetRegistry/Manager, AudioEngine)
|
Layers -> EditorLayer | RuntimeLayer
|
Scene (EnTT) -- PhysicsWorld (Jolt)   ScriptRegistry
|                                     |
|                                     | contact events
|                                     | v
|                                     | scripts --> AudioEngine (miniaudio)
|
RenderGraph <- BuildFrame -> PostProcess
|
Renderer2D / Renderer3D / DebugRenderer
|
RHI -> Platform/Vulkan | Platform/OpenGL
```

Dependency runs strictly downward. The `Scene` knows nothing about the renderer; the renderer reads the scene and never writes it. The RHI knows nothing about scenes, meshes or materials — it knows buffers, textures and pipelines. Two consequences worth stating because they are what the layering buys:

- The test suite can build scenes, simulate them and assert on them **without a GPU**, because nothing in the scene layer needs a device.
- A second backend was added without touching a single renderer file, because everything above the RHI is written against the interface rather than against Vulkan.

3.2 The module boundary — engine as a DLL

The engine builds as `RageV.dll`. The editor, the runtime, the test tools and each project's game module link against it. This is what makes per-project C++ scripting possible without an engine rebuild, and it imposes two constraints that are easy to violate silently:

Static data read through an inline accessor does not cross a module boundary. The consumer links its own copy. Anything that must be a singleton across the boundary needs an exported out-of-line accessor.

A translation unit containing only static registrars is dropped from a static library. It no longer bites the engine, which is linked from object files, but it is why built-in scripts are still registered by an explicit function call — and it would bite again the moment anything moved back into a `.lib`.

A related trap, found late and worth generalising: **a tool must not link a vendored library the engine already links privately.** The RHI smoke-test tool linked GLFW a second time and called `glfwInit` itself, so the executable and the DLL each held their own GLFW globals. The window was real and the engine's copy had never heard of it, so Vulkan reported a null `HWND` and OpenGL reported that it could not load GL functions — *one cause, two symptoms that look completely unrelated*.

3.3 The three "one description" systems

Three places in the engine define something once and generate everything else from it. Each replaced two or three hand-maintained copies that had already drifted at least once. This is the single most reusable idea in the codebase.

Component reflection

Each component is described once — its name, its fields, their types and editor hints:

```
// Order is the serialization order and the inspector order, so it is
// fixed here rather than emerging from a hash map.
ComponentDesc desc;
desc.Name = "ParticleEmitterComponent";
desc.DisplayName = "Particle Emitter";
desc.Fields = {
    Field<&ParticleEmitterComponent::Rate>("Rate", Drag(1.0f, 0.0f, 10000.0f)),
    Field<&ParticleEmitterComponent::ColorStart>("ColorStart", Color()),
    Field<&ParticleEmitterComponent::SizeCurve>("SizeCurve",
        Named("Size curve", AssetRef(AssetType::Curve, "..."))),
    Field<&ParticleEmitterComponent::Blend>("Blend", Enum(kParticleBlendNames)),
};
Bind<ParticleEmitterComponent>(desc);
```

From that one description the engine derives **the scene file format, the inspector UI, the undo system's field-edit commands**, and **the C# component bridge** — C# reaches components by registry name with values as text, so a component gaining a field is visible from C# the moment it exists. Nothing here can go stale, because there is nowhere for it to go stale *to*.

The cost of a type-erased system is that type errors become runtime errors. One of them was worth converting back into a compile error:

```
else if constexpr (std::is_enum_v<T>)
{
    static_assert(sizeof(T) == sizeof(int),
        "A registered enum field must be int-sized: reflection reads and "
        "writes it through an int*, so a narrower underlying type is a "
        "silent memory stomp. Remove the ': uint8_t' from this enum.");
    return FieldType::Enum;
}
```

An `enum class : uint8_t` compiled, drew correctly, and wrote three bytes past the end of the field. It is now a build error naming the offending enum.

The RHI

One interface, two implementations; discussed in §5.1.

Asset handles

One 64-bit identity per asset, one registry, one `.meta` sidecar. Every reference anywhere — scene files, prefabs, components, the C# bridge — is that handle. When particle curves were added as a new asset type, the inspector, the serializer and the C# bridge all worked immediately, because a curve reference is an `AssetHandle` and the system already knew what those were.

4 The frame

4.1 Fixed simulation, free rendering

```
clamp frame time (0.25s)
InputMap::Update()           once per frame
for each fixed step:         zero or more
    Layer::OnFixedUpdate(dt)  scripts, then physics
    InputMap::EndFixedStep()
alpha = accumulator / dt
BeginFrame -> Layer::OnUpdate(ts) -> ImGui -> EndFrame
```

Simulation advances in fixed steps; rendering does not. Above 60 Hz a frame commonly runs *zero* steps, which is exactly why rendering must interpolate rather than read the simulation directly. The renderer blends the last two simulation states by `alpha`.

This split has a consequence that shapes the whole scripting API: **presentation belongs on the frame, simulation belongs on the step**, and every system has to pick one deliberately. In `Scene::OnUpdateRuntime` — the per-frame pass — each choice carries its reason:

```
// On the frame rather than the fixed step, for the same reason the audio
// positions are: an animation is presentation, and pinning it to the
// simulation rate would make it stutter at any other frame rate.
UpdateAnimators(ts);

// Per frame, not per step: this is where the blend between the last two
// simulation states is applied, and it is the frame that needs it.
if (m_Physics)
    m_Physics->SyncTransforms(*this, Application::GetInterpolationAlpha());

// After the sync, so a sound on a simulated body is placed where it is
// being drawn rather than one step behind it.
UpdateWorldTransforms();
UpdateAudio();

// Presentation too: nothing collides with a particle and nothing scores
// one, so they move at whatever rate the display shows them.
Particles::System::Update(*this, ts.GetSeconds());
```

Scripts are currently the one system still pinned to the fixed step, which is a known gap: a camera moved from a fixed-step callback updates on one frame in four at 240 Hz, against a world that interpolates on all four — differential stutter, which reads worse than no smoothing at all.

4.2 The render path, pass by pass

(prefilter)	->	roughness levels, 36 small renders, ONCE per environment
(shadow maps)	->	4 cascades + a map per casting spot and a cube per point, all scene renders, BEFORE the graph opens
(probe faces)	->	probe cube 0..6 scene renders, BEFORE the graph opens
Scene	->	SceneHDR RGBA16F, linear, no tone curve
meshes		reflect the environment cube
sky		after the meshes, depth-tested, no depth write
Transparent	->	accum + reveal weighted-blended particles (only if used)
ResolveTransparent	->	SceneHDR composites the two buffers back
Overlay	->	SceneHDR colliders, preserved load, depth-tested
Bloom prefilter	->	Bloom0 13-tap, Karis weighted, clamped, soft knee
Bloom down 1..4	->	Bloom1..4 13-tap, halving each time
Bloom up 4..1	->	Bloom3..0 3x3 tent, ADDITIVE, accumulates in place
Tonemap	->	LDR exposure, + bloom, ACES, 1/2.2
FXAA	->	Output on the tone-mapped image, not the linear one

Several ordering decisions here are not arbitrary and are easy to get wrong:

- **The sky is drawn after the opaque meshes**, depth-tested with no depth write. Drawing it first wastes fill on every pixel the scene covers; the depth test has nothing to test against until the meshes have written depth.
- **FXAA runs on the tone-mapped image**, not the linear one. Edge detection on unbounded HDR values finds edges that are not perceptually there.
- **Shadow maps and probe captures happen before the graph opens**, because each opens render passes of its own and a pass cannot be nested inside another.
- **Shadows are fitted to the camera they will be drawn with**, since cascades are fitted to a frustum — so the editor's two viewports fit their own.

4.3 The render graph

A pass declares what it writes and what it samples; the graph pools the targets and executes in declaration order. A pass is two lambdas — one that declares, one that draws:

```
graph.AddPass("Scene",
    [&](RGPassBuilder& builder)
    {
        // Attachment 0 only: every pipeline that draws the scene declares
        // one colour format, and a pass binding three would require all
        // of them to declare three.
        builder.WriteAttachments(sceneHDR, { { 0, desc.ClearColor } });
        builder.SetClearColor(desc.ClearColor);
    },
    [draw = desc.DrawScene](RGPassContext& context)
    {
        if (draw)
            draw(context);
    });
```

Targets are pooled by shape, so eleven passes use seven targets. The graph also refuses malformed frames at compile time rather than producing an undefined picture: a pass binding an attachment its target does not have, or a target created and never written, are both errors.

The payoff is concrete. Weighted-blended transparency needed a target with three colour attachments over one shared depth image, with the scene pass binding attachment 0 and the transparent pass binding 1 and 2 — a structural change to the frame that amounted to *two new graph capabilities and two pass declarations*, and touched no existing pass.

5 Rendering engineering

5.1 Designing an RHI over two very different APIs

Vulkan and OpenGL disagree about almost everything that matters: who owns synchronisation, whether resources are bound or described, how memory is allocated, and which way up an image is stored. An abstraction over both is an exercise in choosing which disagreements to hide and which to expose.

What to abstract

The nouns, not the verbs of one API. `RHIDevice`, `RHIBuffer`, `RHITexture`, `RHIPipeline`, `RHIShader`, `RHIResourceSet`, `RHISampler`, `RHICommandList`. These exist in both worlds even when only one names them.

What to expose rather than hide

Anything whose cost differs by an order of magnitude between the two. Frames in flight is the clearest example: it is a Vulkan concept, OpenGL has no equivalent, and hiding it would have meant either paying Vulkan's synchronisation cost on OpenGL or silently writing buffers the GPU was still reading. It is exposed, and OpenGL implements it honestly — which found a real bug, see §5.2.

What to design at the level of intent

Buffer barriers are the good example. Vulkan wants an access mask and a stage; OpenGL wants a memory-barrier bit. Neither vocabulary translates. The RHI takes a `BufferSync` describing the *use* — "was written by compute, will be read as an index buffer" — which both backends can implement without lying.

One operation that is worth its cost

```
// Records a one-shot command list and blocks until the GPU has finished it.  
// Outside the frame loop entirely: no swapchain image is acquired and  
// nothing is presented.  
//  
// It stalls, by construction -- that is what makes it usable for work whose  
// result is needed immediately, and what makes it wrong on the frame path.  
virtual void ExecuteImmediate(const std::function<void(RHICommandList*)>& record) = 0;
```

Asset upload uses it, one-off compute dispatches use it, and — the reason it earns its place — it is what lets the headless test suite exercise real GPU work without a frame loop to borrow.

5.2 Frames in flight, and everything that follows

The CPU records frame $N+1$ while the GPU executes frame N . Everything the GPU might still be reading therefore needs one copy per frame in flight, and everything destroyed needs to outlive the frame that used it. Three rules follow, all learned the hard way.

Every backend needs more than one frame in flight and a fence between them. OpenGL originally reported one and had no fence, so subsystems rewrote per-frame buffers the GPU was still reading — a static scene rendered three different images in six frames. Fixing it cost 6% and bought correctness.

A resource set holds one descriptor set per frame in flight, and commit writes only the current frame's. Committing once at creation populates one and leaves the rest never written: validation catches it on

frame two, a release build renders garbage.

The third is subtler and took a purpose-built repro to find. Deletion is deferred to a per-frame-slot queue — but entity destruction happens in the simulation phase, *before* `BeginFrame`, so a deleter pushed there landed in the slot `BeginFrame` was about to flush in the same iteration:

```
void Push(std::function<void()> deleter)
{
    if (!DeviceAlive || PerFrame.empty()) return;

    // Out-of-frame pushes are slotted by the most recently *submitted*
    // frame, not the current index -- otherwise they land in the slot
    // about to be flushed, behind a fence covering a frame from two
    // submissions ago. Deferred by zero frames, in every process.
    const uint32_t count = (uint32_t)PerFrame.size();
    const uint32_t slot = InFrame ? FrameIndex : (FrameIndex + count - 1) % count;
    PerFrame[slot].push_back(std::move(deleter));
}
```

The GPU usually won the race, which is why it read as a rare runtime flake. It was proven fixed with a churn scene destroying ~180 materials a second under GPU load: **10 validation errors in 4000 frames before, zero in 12000 after.**

5.3 Batching and instancing

Meshes sharing a mesh handle, material and pipeline state are drawn as one instanced call, with per-instance transforms in a storage buffer. Two things about this are worth transferring.

Identical state must be the same object, or a batch key cannot see it. Materials each created their own sampler with identical settings. The batcher compared sampler pointers, saw a difference that did not exist, and silently prevented all lit batching. Sharing one sampler dropped the frame to 60 draws and won 4.5× on OpenGL.

A batch's base instance is a push constant, never `firstInstance`. Vulkan's `gl_InstanceIndex` includes the base offset; OpenGL's `gl_InstanceID` does not. Using the API's own base instance produces a correct picture on one backend and scrambled transforms on the other.

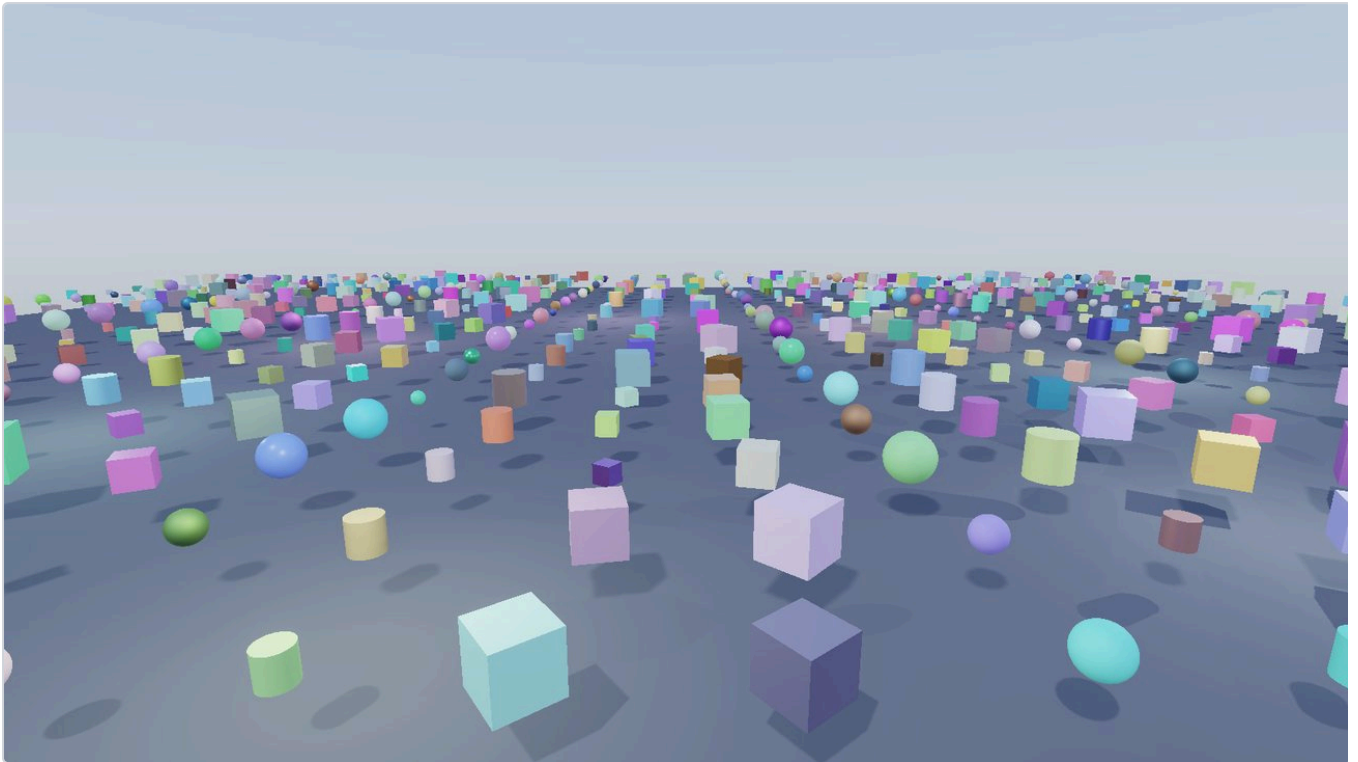


Figure 4. The stress scene — 1000 meshes at 1.88 ms (Vulkan) and 1.59 ms (OpenGL), Release, vsync off. OpenGL winning here is real and not a bug: at this draw count the frame is dominated by per-draw CPU cost, where GL's simpler binding model has less to do.

5.4 Lighting: clustered forward

The view frustum is divided into a $16 \times 9 \times 24$ grid of cells; lights are binned into cells on the CPU; the fragment shader loops only over the lights in its own cell. Directional lights are deliberately *not* binned — they reach every cell, so binning them is pure overhead.

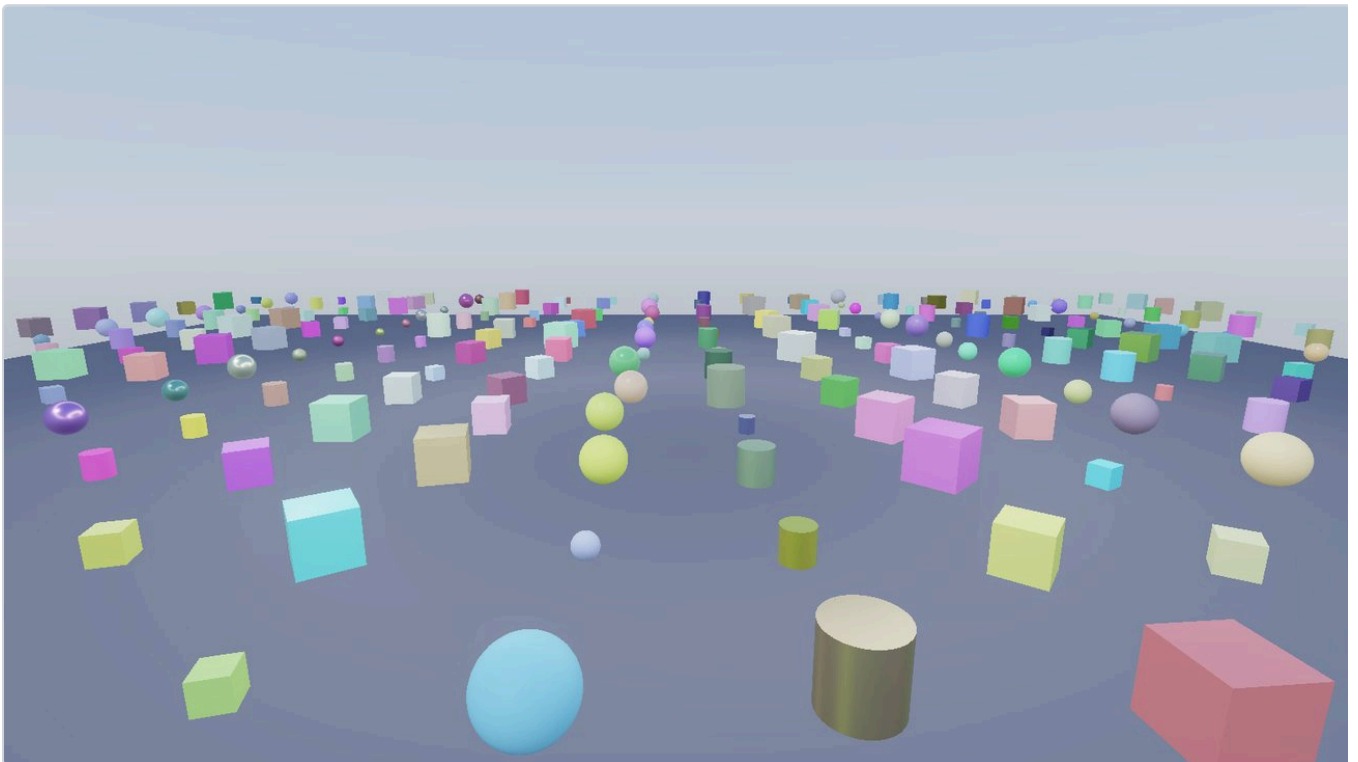


Figure 5. 256 local lights. Clustered: 3.15 ms Vulkan / 3.48 ms OpenGL, against 5.56 / 7.03 ms unclustered, with a pixel-identical image.

Know where your optimisation does not help. Where every light reaches the whole scene, the busiest cell holds all of them and clustering is pure overhead — measured at 5.76 → 6.62 ms, a loss. The benchmark prints "busiest cluster holds N" precisely so this is visible rather than mysterious, and the scene generator takes a `--light-reach 0` flag to construct the pathological case on purpose.

5.5 Shadows

Cascaded shadow maps for directional lights (four cascades fitted to the camera frustum), a single map per casting spot light, and a cube map per casting point light. Three rules that are each a day lost if missed:

- **Whatever poses the lit pass must pose the depth pass.** Otherwise a character walks and its shadow stands still.
- **Nothing is culled in the shadow pass.** An object outside the camera frustum can still cast into it.
- **OpenGL needs `glClipControl(..., GL_ZERO_TO_ONE)`** or every shadow comparison passes and the scene is uniformly lit.

5.6 Image-based lighting and reflection probes

An environment cube map is convolved once into a diffuse irradiance cube and prefiltered into six roughness levels, and a BRDF lookup table is generated. Reflection probes capture the scene into a cube map from a point; the scene picks the nearest probe whose influence radius contains the camera, falling back to the sky.

Mip generation belongs in the command buffer that wrote mip 0. `GenerateMips` submitted its own and waited, so a probe's chain was built from an empty mip 0 and its metal read black for six frames. A bug that heals itself reads as a warm-up; it took a per-frame luminance measurement to show a hard step at frame 7 rather than a gradual fill.

5.7 Post-processing

Bloom (13-tap Karis-weighted downsample, 3×3 tent upsample accumulating in place), exposure, ACES tonemapping and FXAA. One invariant here is the single most instructive bug in the codebase, because of how long it hid:

```
// A fullscreen pass reads one render target and writes another, and the two
// backends do not store a rendered image the same way round.
//
// Vulkan's first framebuffer row is the top of the image; OpenGL's is the
// bottom. The RHI hides that everywhere else by giving Vulkan a negative-
// height viewport, so geometry lands in the same place on both -- but that
// fixes where a fragment *writes*, not where a texture coordinate *reads*.
//
// An even number of passes hides it, which is why this survived: with
// anti-aliasing on, the scene goes tonemap -> FXAA and comes out the right
// way up. The bloom chain has an odd number, and its contribution was being
// added upside down -- visible only once something was bright enough to
// bleed, as blobs mirrored about the middle of the image.
local.FlipY = s_Data->Device->GetBackend() == Backend::Vulkan ? 1.0f : 0.0f;
```

It then bit a *second* time, years of commits later, and the reason is the transferable part. The invariant had been written down as "...and nothing else samples a render target", which stopped being true the moment the transparency resolve pass was written — a fullscreen pass living in the particle renderer rather than the post-

processing file, so it inherited none of this. **A rule stated in terms of where code lives will be violated by code that lives somewhere else.** It is now stated in terms of behaviour: anything that samples a render target and writes another owes the flip.

5.8 Transparency, and why it is the hardest easy problem

Opaque geometry is order-independent because the depth buffer sorts it. Transparency is not, and everything else follows from that.

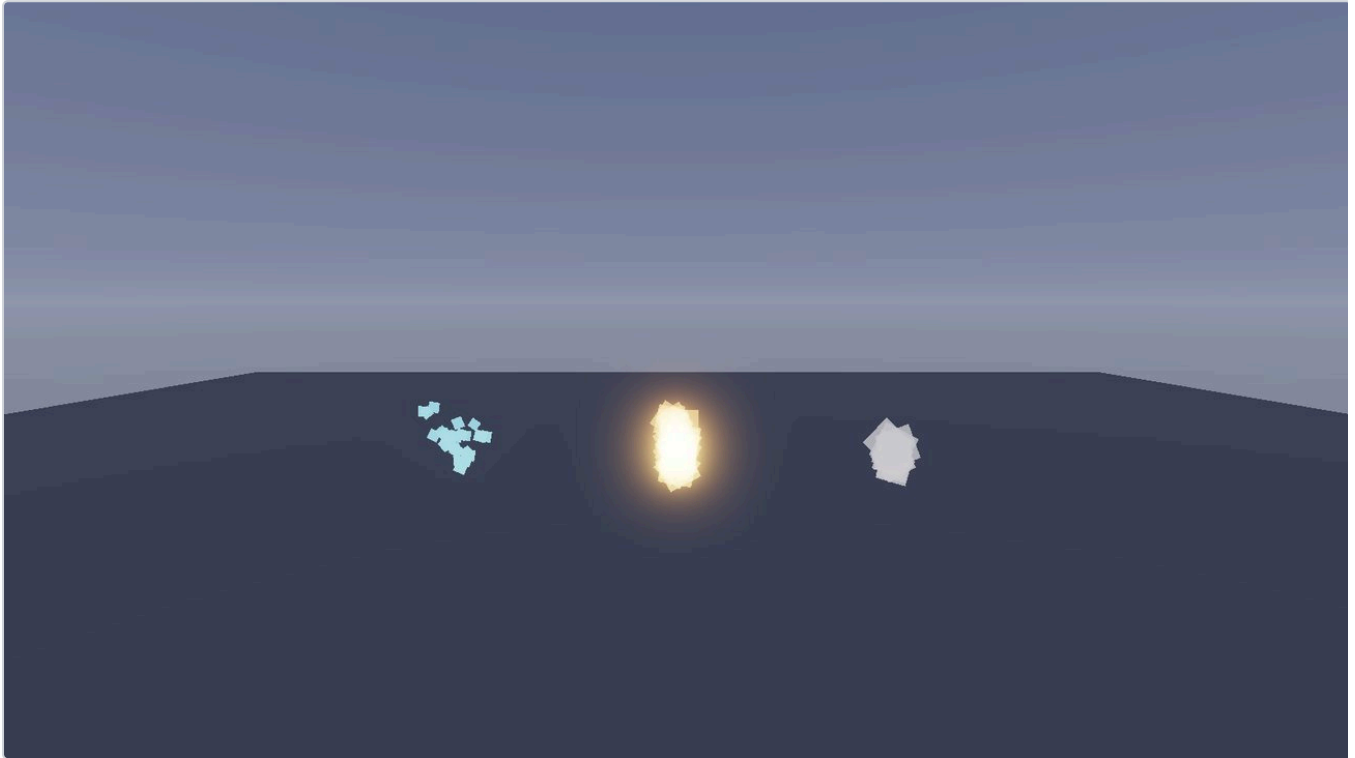


Figure 6. The three answers, in one scene. Left: a 2D flat-facing fountain. Centre: additive fire. Right: weighted-blended smoke.

Mode	How	Correct when
Additive	Sums light; order cannot affect a sum	Always — for things that <i>are</i> light: fire, sparks, flashes
Alpha	Sorted back to front	Exact, but needs a sort — impossible for a GPU-simulated pool without a readback
WeightedBlended	Two order-independent accumulators plus a resolve	Approximate, needs no sort, and the approximation is good for dense plumes

Weighted-blended OIT (McGuire & Bavoil) accumulates premultiplied colour weighted by a depth-dependent term into one buffer, and coverage into another; a resolve pass divides them. The technique lives or dies on that weight function, and validating it is a lesson in how to test a picture.

How to validate something whose failure mode is "looks fine"

The weight was taken from a well-known published snippet and eyeballed once. To check it, a scene was built whose correct answer is *arithmetic*: three emitters at 2, 20 and 200 units, one motionless particle each, red/green/blue at alpha 0.5, every spatial quantity scaled by distance so all three subtend the same angle and land exactly concentric. Compositing 0.5 red over 0.5 green over 0.5 blue is 4:2:1 — a red-dominant pixel —

and a single sorted CPU emitter produces that by construction. So the reference is not a stored image but a render that is *correct by definition*.

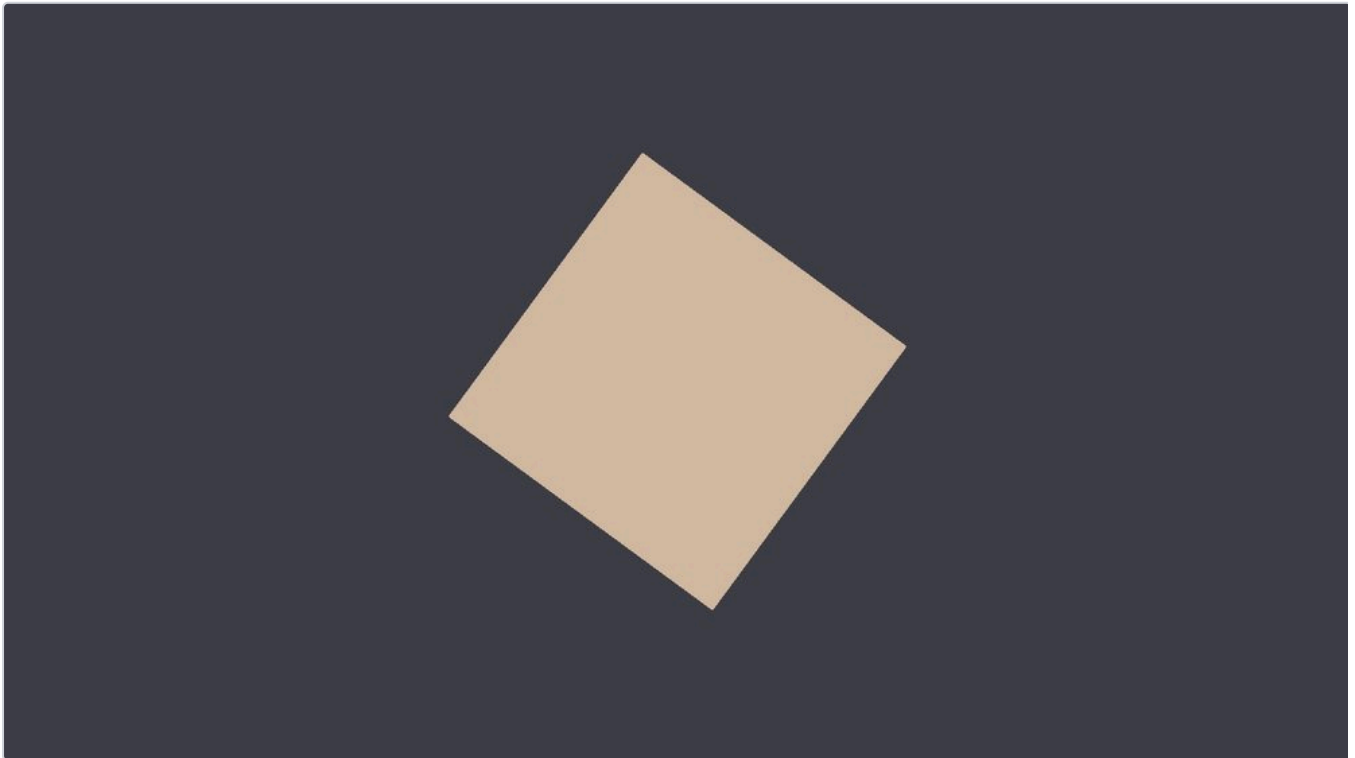


Figure 7. The instrument. Three identical emitters at 2, 20 and 200 units, drawn concentric. Rendered as sorted alpha this is the ground truth; rendered weighted it exposes exactly how much the depth weight is doing.

It found the weight was **a constant**. The published snippet uses `gl_FragCoord.z`, which is non-linear: with the sample scenes' near plane of 0.01 it spans 0.995 to 1.0 across the entire world, so the depth term varied by 4% from arm's reach to the horizon — and a `1e8` multiplier put the product 39× over the clamp ceiling at every distance anyway. Since the resolve divides accumulated colour by accumulated alpha, *a constant weight cancels exactly*: the output was an unweighted average that still looked like plausible transparency. Proven by replacing the whole expression with the literal `1.0` and getting a pixel-identical image.

A formula whose output is later normalised must be tested for whether it still *varies*, not whether it looks sensible. A constant is the one value that hides perfectly inside any normalisation.

The replacement falls as $1/\text{distance}$ on linear view depth. That is not the paper's recommendation, and the reason is a property the error table does not show: under the standard clamp, the paper's equation 9 only discriminates between 1.5 and 100 units and clamps flat outside that band — it draws the 20-unit and 200-unit layers identically. $1/d$ discriminates from 0.03 to 9000 units, so ordering never collapses, and the range it buys is at the *near* end, which is where a camera standing inside a smoke cloud actually lives.

5.9 Compute, and moving a simulation to the GPU

Particles simulate on the CPU by default and on the GPU per emitter, with a switch that allocates nothing: the compute pipeline is built once, and an emitter's buffers are cached by UUID and retained while the emitter *exists*, not while it is using them. Measured at ~16k particles: **Vulkan 3.30 → 1.37 ms, OpenGL 3.80 → 1.58 ms.**

Two findings generalise well beyond particles.

A persistently-mapped host-visible buffer written by a compute shader is a synchronisation point on OpenGL. The state buffer was host-visible so a rare seed could be a memcpy. That made the GPU path *slower than the CPU one* — 6.9 ms against 3.3 — and stopped GPU timestamps resolving at all. Device-local, seeded through staging, fixed both. The first hypothesis (barrier interleaving) was wrong; only measurement found it.

The second is about keeping two implementations of one thing honest. When curve-driven ramps arrived, the naive design would have given the compute shader its own copy of the rule deciding whether a curve or a start/end pair wins per channel. Instead the CPU resolves the ramps once per emitter per frame and uploads the *answer* — 64 samples — so the shader has no opinion to disagree with:

```
// The ramps, already resolved on the CPU by Particles::Evaluate.
//
// This shader is handed the answer rather than the ingredients: it has no
// idea whether a channel came from a curve asset or a start/end pair, which
// is deliberate. That rule lives in exactly one place, and a shader that
// cannot express it cannot disagree with it.
vec4 ColorRamp[64];
vec4 SizeRamp[64];    // x only
```

Verified by measurement, and the measurement found something else: under **Additive** the CPU and GPU paths agree to 0.10 of 255 on both backends; under **Alpha** a curved emitter differs by 2.7, seven times the run-to-run noise. That gap is not the ramp — switching to an order-independent blend collapses it — it is the GPU's inability to sort within an emitter, which *curves make worse* because they widen how much particles differ in colour and opacity.

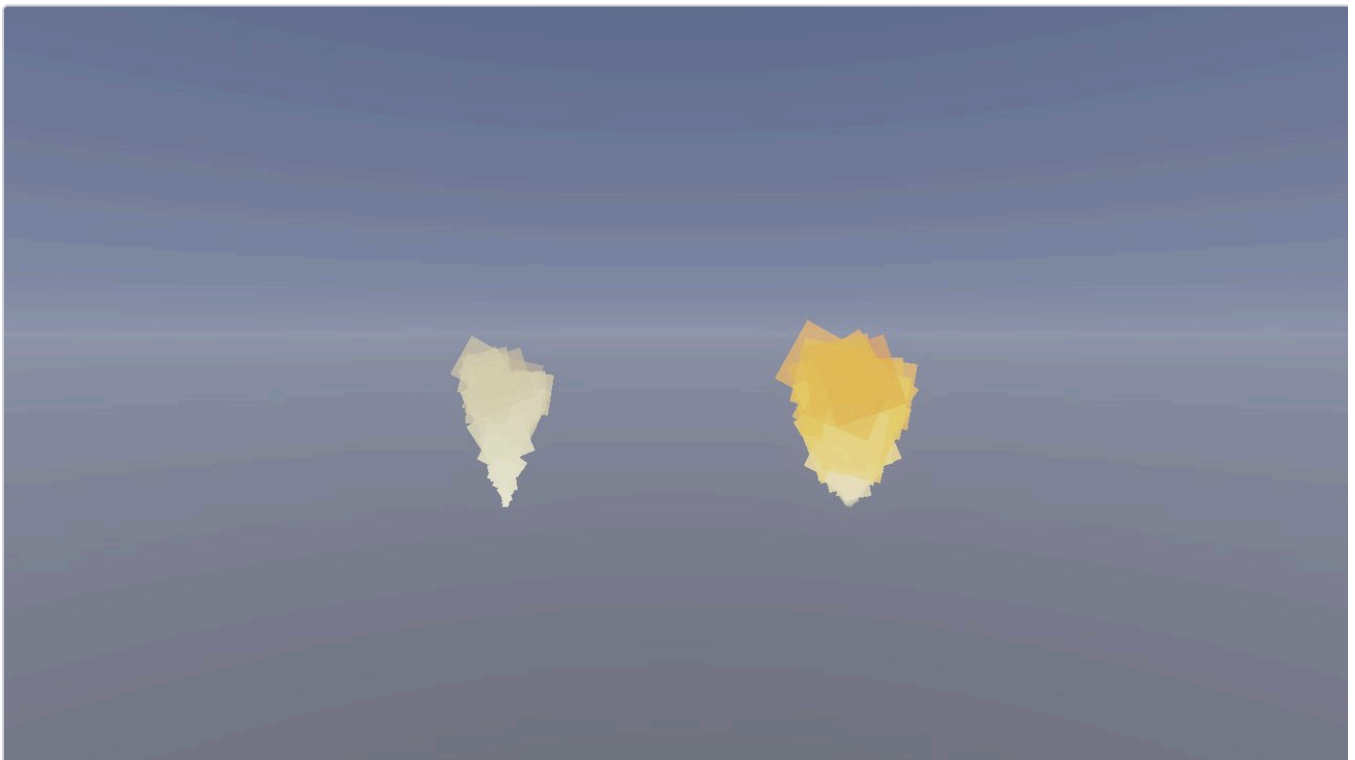


Figure 8. The same emitter twice. Left: size, colour and alpha as start/end pairs. Right: the same three driven by curves — the gradient passes *through* orange rather than crossfading past it, the size curve rounds it out early, and an alpha curve fades it in *and* out, which two endpoints cannot express at all.

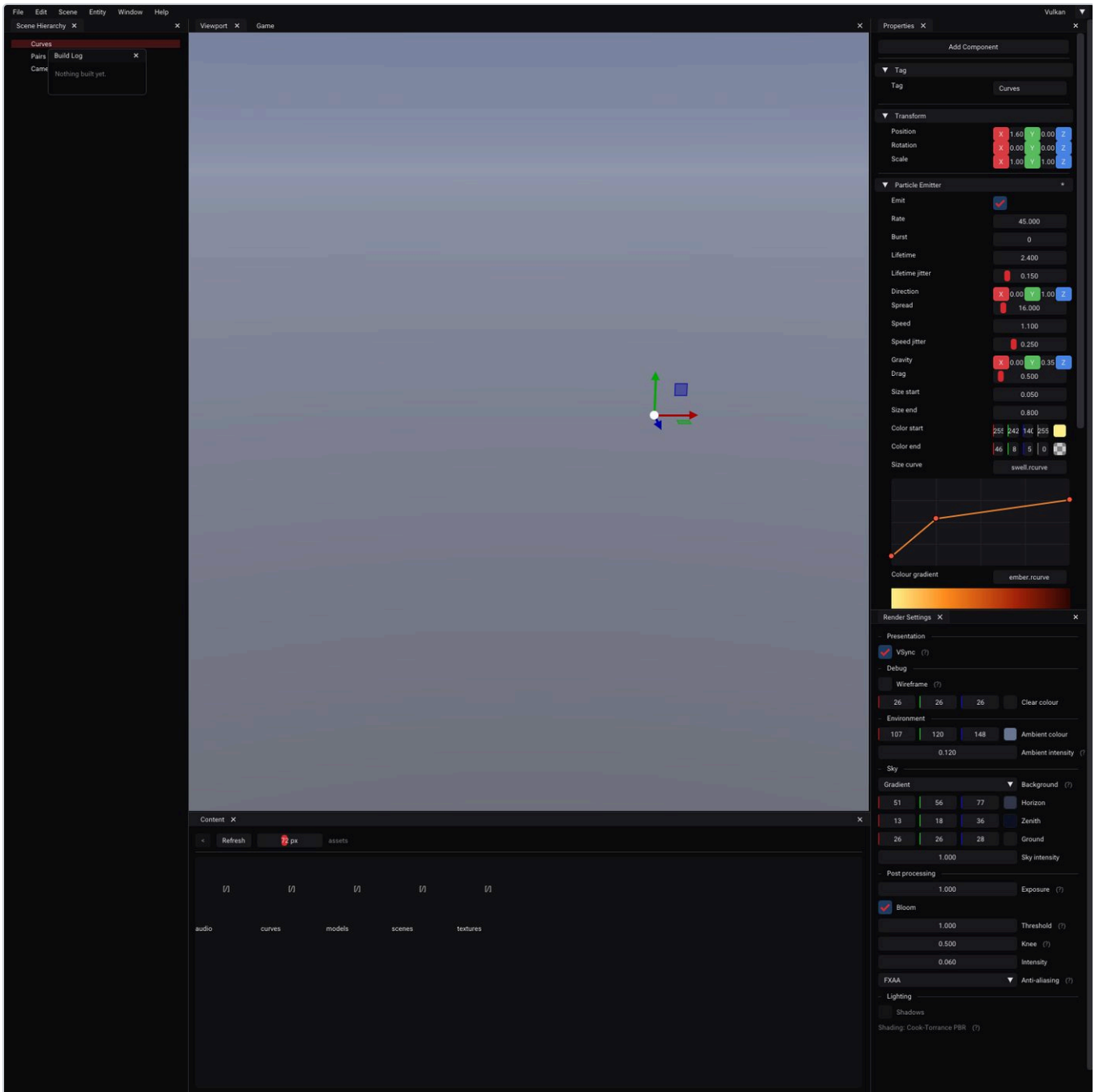


Figure 9. Curves are assets, edited inline under the field that references them — a graph with draggable points for a scalar ramp, a stop strip for a gradient. Because a curve reference is an ordinary `AssetHandle`, the inspector, the scene format and the C# bridge all supported it without changes.

6 The systems around the renderer

6.1 Scene, entities, reflection

EnTT provides the storage. On top of it: a UUID on every entity, a relationship component holding only the parent link, world transforms recomputed in one top-down pass per frame, and the component registry of §3.3.

World transforms are deliberately **not** a dirty-flag cache:

```
// Recomputed unconditionally by Scene::UpdateWorldTransforms in one
// top-down pass per frame.
//
// Deliberately not a dirty-flag cache. A flag has to be set at every write
// site -- the inspector, the gizmo, scripts, the serializer, and everything
// added later -- and a single missed one leaves an object silently
// rendering in the wrong place. Recomputing is O(n) at scene scale and
// cannot be got wrong.
Mat4 World{ 1.0f };
```

This is a good example of choosing the boring implementation on purpose. The cache is faster and the cost of one missed invalidation — an object in the wrong place, sometimes — is far higher than the saving.

6.2 Assets

Handles, a registry mapping handle to path, `.meta` sidecars, and a manager that loads and caches by handle. Two conventions worth copying:

- **Failures are cached.** A missing texture caches as null so a broken path is not reopened sixty times a second.
- **Never return null for a valid handle where a value will do.** A missing curve answers an *empty* curve that evaluates to its fallback, so the caller samples a value rather than every call site remembering a branch. Only an *invalid* handle answers null — which keeps "no curve authored" distinguishable from "curve failed to load".

6.3 Scripting in two languages

C++ scripts live in a project's `Source/` and build into a game module DLL the engine loads. C# scripts live in `Scripts/` and build into an assembly loaded from bytes into a collectible load context. Both hot reload; building while playing stops the scene, swaps both languages, and resumes play on the new code.

The interop is an append-only function-pointer table with an explicit protocol version in both the native and managed headers. The one structural asymmetry is worth calling out because it is inherent rather than incidental: a template cannot cross a language boundary, so where C++ says `GetComponent<LightComponent>()`, C# reaches components **by registry name with values as text** — the same names and text forms the inspector and the scene file use. That trade is what keeps the C# surface from going stale when a component gains a field.

6.4 Physics and audio

Jolt, stepped after scripts within the fixed step, with contact events delivered after the step that produced them. Colliders are authored in local units and multiplied by the transform's absolute scale — worth knowing, because authoring the world radius directly produces a collider that is silently $\text{scale} \times$ too large.

Audio is miniaudio, with positional sources updated per frame *after* transform sync, so a sound on a simulated body is placed where it is being drawn rather than one step behind it.

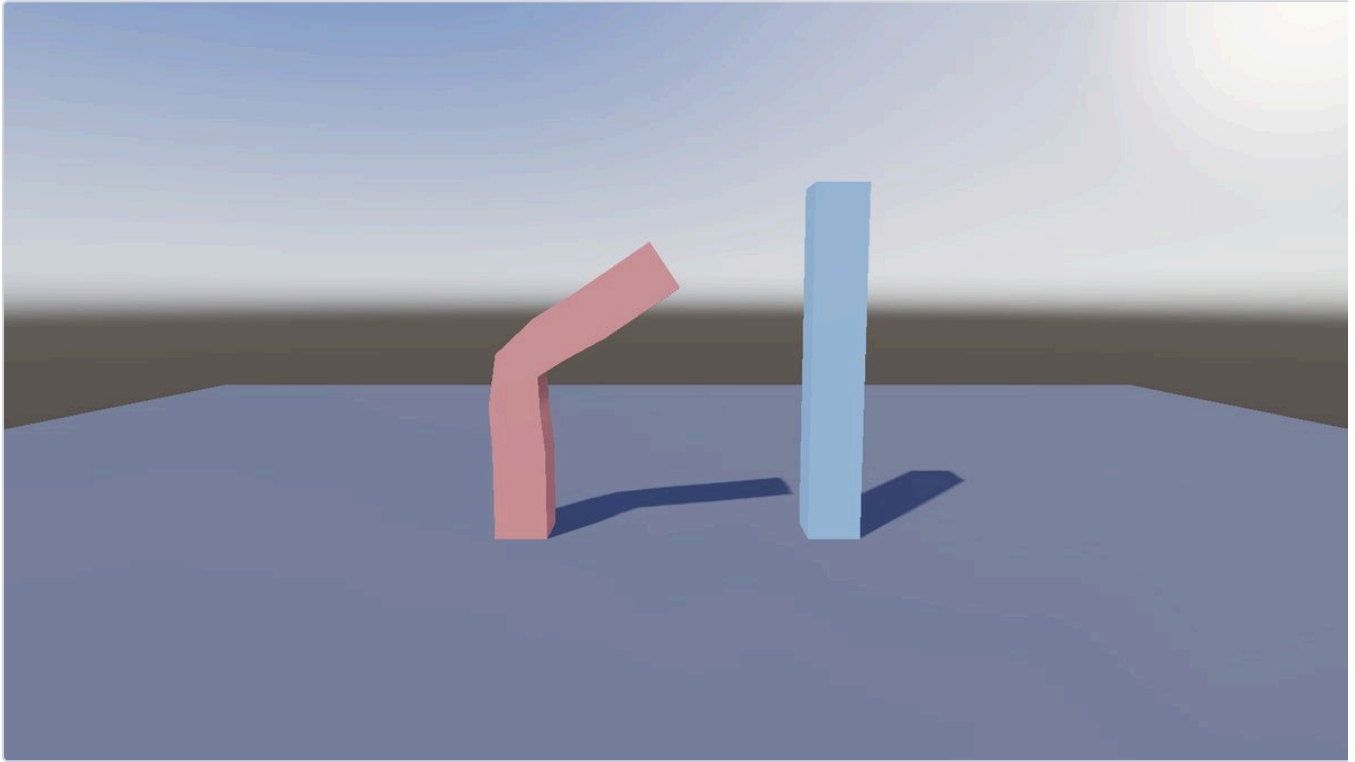


Figure 10. GPU skinning. The invariant that catches the most bugs at once: a skeleton at its bind pose must produce an identity skinning matrix for every bone — one check that catches wrong inverse binds, wrong compose order and wrong space simultaneously.

7 What experience taught

Every entry below was a real bug that cost real time, and most of them fail silently rather than obviously. They are grouped by the kind of mistake rather than by subsystem, because the kind is what transfers.

Things that are two things pretending to be one

- **A swapchain is retired, not destroyed and replaced.** Pass the old handle as `oldSwapchain` and destroy it after the new one exists — `vkDeviceWaitIdle` covers the queues, not the presentation engine.
- **A cache holding one object under two keys must be cleared as one.** The texture loader keyed by path and the IBL filter keyed by pointer were cleared at different times; changing project kept every old environment map alive.
- **Identical state must be the same object (§5.3).**

Things that are silently the wrong size

- **A vertex attribute's reflected format must carry its vector width.** The reflection returned a one-component format for every integer, so a `uvec4` of joint indices was 4 bytes where the buffer held 16, and the mesh drew as a spray of triangles.
- **A registered enum field must be int-sized (§3.3).**
- **A skinned mesh needs one bone matrix per bone, never a short run** — it reads into the next character's.
- **A descriptor pool is a chain, not a ceiling.** 1000 meshes segfaulted a fixed 2000-set pool.

Things that differ between backends and look fine on one

- **Fullscreen passes and the V flip (§5.7)** — and an even number of passes hides it.
- **Base instance (§5.3).**
- **Clip control on OpenGL (§5.5).**
- **`glDrawBuffers` is persistent framebuffer state**, and `glClearBufferfv` indexes the draw-buffer list rather than the framebuffer. Binding attachments 1 and 2 means clearing indices 0 and 1.
- **`independentBlend` is a Vulkan device feature.** Different blend state on different attachments is not free.

Things about lifetime

- **Deferred deletion must be slotted by the frame actually submitted (§5.2).**
- **Never rewrite a bound descriptor set.**
- **`Layer`'s destructor must stay virtual.** Without it only `~Layer` ran and every derived member leaked — the editor's scene, its render targets, every material in them. Undefined behaviour that survived months because nothing looked wrong.
- **Wait for the device to idle before destroying anything**, not partway through.

- **Verify shutdown by exiting, not by killing.** Every one of these was invisible while runs were terminated rather than closed.

Things about measurement

- **A draw count is not a frame time** (§2.5).
- **A bug that heals itself reads as a warm-up** (§5.6).
- **When a timing bug will not reproduce, failing to reproduce it is the evidence** — stop asking what you broke and go find the order-dependent hazard in that path.
- **Ask whether a defect predates your change.** Stashing and rebuilding takes ten minutes and regularly turns "my regression" into "already there".
- **Benchmark runs taken far apart are not comparable.** Re-running an unchanged baseline forty minutes later drifted 1.3% with no code involved.

8 Verification: how any of this is known to work

A renderer is uniquely hostile to testing: its output is a picture, its failures are often plausible pictures, and the usual unit-test instinct produces tests that pass while the screen is blank. This is the methodology that emerged.

The bar, in full

Every change is verified against all of the following before it is committed:

1. **888 automated checks, on both backends, exit code 0.** Not "no failures printed" — the exit code, because a harness that crashes halfway prints no failures.
2. **Zero validation-layer messages**, with validation explicitly enabled. It is off by default because it costs ~1.5 ms/frame and made Vulkan read as slower than OpenGL; a run without the flag reports zero messages whether or not there were any.
3. **The pixels, via the engine's own screenshot capture** — not a draw count.
4. **Both backends' frames compared against each other**, not merely each on its own. A Vulkan-only bloom flip survived an entire roadmap phase of clean runs because nothing in the scene was bright enough for a mirrored contribution to show.
5. **All three build configurations** for anything the editor displays.

Test the property, not the plumbing

The distinction that makes GPU testing tractable. A test that asserts a dispatch was recorded proves nothing. A test that runs 100 elements through a 64-wide workgroup — so the tail group runs past the end and the shader's bounds check is actually exercised — and then checks every element, proves the thing you care about.

Compare against something correct by construction

The strongest tool in the box, and the one that found two shipped bugs in an afternoon (§5.8). Do not compare a render against a stored reference image, which rots and which nobody can debug. Compare it against *another render* whose correctness is guaranteed by construction — a sorted single emitter is exactly correct, so it is the reference for an unsorted approximation of the same particles.

Reintroduce the bug to prove the test fails

A regression check that has never failed is a hypothesis. Both transparency bugs were deliberately put back to confirm the new check catches them and names them correctly. The same discipline was applied to the enum compile-time assert.

Build the repro before the fix

The deferred-deletion race was a rare flake until a scene was built to destroy ~180 materials a second under GPU load. Then it was 10 errors in 4000 frames, reliably — and therefore provably fixed at zero in 12000.

Automate the eyeball where you can

Some checks genuinely need a picture, but "a picture" can still be reduced to a number. The transparency regression check renders the same particles two ways and compares *silhouettes* and channel ordering, which catches both a vertical flip and a weight that has stopped discriminating — neither of which a human reliably notices in a still of smoke.

9 Where it stands, and what comes next

Complete

Phases 0 through 5, the game-module arc, and a shipped mini game. The renderer, scene, asset, scripting, physics and audio systems described above are all in place and verified on both backends.

In progress

Phase 6. Particles are done — CPU and GPU simulation, three blend modes, and asset-backed curves for size, colour and alpha with an inline editor.

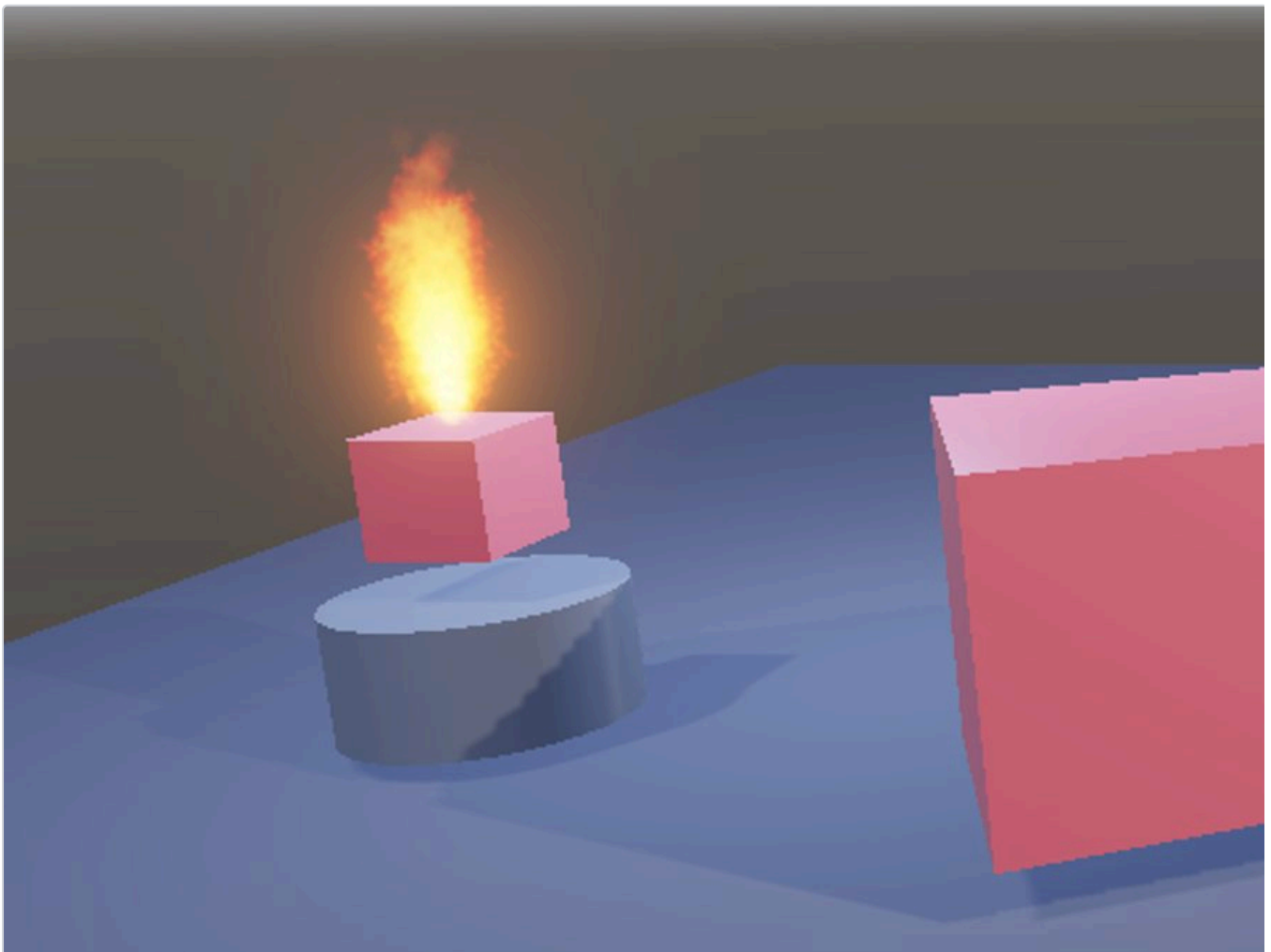


Figure 11. The current end of the road: a textured, curve-driven, additive flame burning on the sample scene's pedestal, in local space so it rides whatever carries it.

Open, in priority order

1. **Text and game UI.** The largest gap by distance, and the one the mini game hit hardest — it could not display a score, a label or a prompt, and communicated entirely through a light and four sounds.
2. **A per-frame script hook.** The second largest, and analysed in §4.1: every other presentation system already runs per frame, and scripts moving on the simulation step actively fight the interpolation.

3. **GPU alpha self-sorting.** A bitonic sort in compute keyed on view depth; because the default pool is 1000 particles, anything up to 2048 sorts inside a single workgroup in shared memory — one dispatch. The measurement in §5.9 is the argument for it.
4. **Particle collision and sub-emitters**, both scoped and deliberately deferred.

What a reader should take away

Not the feature list. Three habits:

- **Order the work by what other work will be described in terms of.** Identity, serialization, reflection and the two abstractions came first, and everything else was cheaper because of it.
- **Build something real with the engine before extending it.** One small game found six holes that no amount of internal testing had.
- **Measure, and construct references that cannot be wrong.** Most of the hard-won entries in §7 were found by a measurement, and most of the ones that took longest were the ones where the picture looked plausible.

A Appendix — repository layout, build, tools

Layout

Path	Contents
RageV/src/RageV/	Engine: Core, Scene, Renderer, Asset, Physics, Audio, Animation, Particles, Managed, Project, Scripts, Math
RageV/src/Platform/	Vulkan and OpenGL RHI implementations, Windows platform
RageVEditor/	Editor application, panels, and the engine's shipped shaders
RageVRuntime/	Standalone runtime
RageVScriptCore/	The managed class library C# scripts compile against
SampleProject/ , Knockdown/	A sample project and a complete game
tools/	scenetest , rhismoke , rvdock , rvpack , and Python asset generators
docs/	Handoff, engine notes, roadmap, and the generated developer manual

Build and verify

```
cmake --build build --config Debug

build/bin/Debug/scenetest/scenetest.exe --rhi=vulkan      # 888 checks, exit 0
build/bin/Debug/scenetest/scenetest.exe --rhi=opengl
build/bin/Debug/rhismoke/rhismoke.exe 120 --rhi=vulkan

build/bin/Debug/RageVRuntime/RageVRuntime.exe \
    --rhi=vulkan --validation=on --screenshot=f.png
```

Flags worth knowing

Flag	Purpose
--rhi=vulkan opengl	Backend; also settable in ragev.ini
--validation=on	Validation layers. Off by default, ~1.5 ms/frame
--screenshot= , --screenshot-frame=	Capture a frame and exit cleanly — the basis of visual verification
--benchmark=N	Frame-time statistics with the vsync state printed beside them
--scene= , --project=	Open a specific scene or project without editing configuration
--select=	Editor: open with an entity selected, so an inspector widget can be verified
--fixed-hz=	Run the simulation at a different rate to see what breaks

A last practical note. Running an application without `--screenshot` or `--benchmark` leaves a window open indefinitely. That is not a hang, and both flags exist so that every verification run terminates on its own with an exit code worth trusting.